

ProductionManagementAI — 製造指示一覧のクエリ — テーブル定義書

002_DB — 要件 REQ-020～REQ-027（work-items/WI-003/brief.md）、基本設計 002_BD（docs/en/010_basic-design/002_BD_production-order-list.md）。002_DD と 002_DD-API（docs/en/020_detailed-design/、作成予定）を実装する。

WI-005 による改訂（2026-09-23）：初期データの備考を日本語にした（シードの表を参照）。それ以外の変更はない。

本書はテーブルも列も追加しない。Screen B は 001_DB が定義したスキーマを読む。データベースに必要なのは、絞り込みと既定の並び順を支えるインデックス、定義された件数取得のクエリ、安定したページングの順序、デモ用のデータである。001_DB が products、production_orders、production_order_number_counters について述べていることはすべて有効なままで、ここでは再掲しない。

物理名は 000_DB と 001_DB に従う：snake_case、EF Core の pk_ / fk_ / ix_ / ck_ の制約名。

テーブル一覧

テーブル	物理名	目的	002_DB での変更
製品	products	製品の参照データ（001_DB）	なし — 読み取りのみ。製品の列と絞り込みの選択肢のために結合する
製造指示	production_orders	製造指示 1 件につき 1 行（001_DB）	新しいインデックス 2 つ、デモ用の初期データの行
指示番号カウンター	production_order_number_counters	年ごとの指示番号の連番（001_DB）	初期データの製造指示と整合させるための初期データの行のみ

ER 図とリレーション

001_DB から変更なし — その ER 図を参照。Screen B がたどるリレーションはちょうど 1 つ：

テーブル	関連テーブル	リレーション（1:1 / 1:N / N:N）	FK 列
products	production_orders	1:N	production_orders.product_id

テーブル定義

列の追加、削除、型の変更はない。Screen B が読む列：

production_orders（SCR-002 の読み取りの射影）

項目名	物理名	用途
Id	id	行のリンク先 /production-orders/{id}（REQ-025）
OrderNumber	order_number	表示列、並べ替えキー、部分一致の絞り込み、ページングの同順位の解決（REQ-021～REQ-024）
ProductId	product_id	製品の絞り込み、表示する製品の結合キー（REQ-021、REQ-022）
Quantity	quantity	表示列、並べ替えキー
DueDate	due_date	表示列、既定の並べ替えキー、範囲の絞り込み、納期遅れの印（REQ-021～REQ-023）
Status	status	表示列、並べ替えキー、複数選択の絞り込み（REQ-021～REQ-023）
UpdatedAtUtc	updated_at_utc	表示列、並べ替えキー

notes、created_at_utc、order_year、order_seq、xmin は一覧のクエリで**選択しない** — 射影は明示的なので、画面に表示する以上の幅の行を取得することはない。
products.sku と products.name は製品の列のために結合で選択する（002_BD M-06）。

アクセスパターン

一覧画面はリクエストごとにちょうど 2 つの文を発行する（FN-010）。どちらも EF Core がパラメータ化されたクエリとして組み立て、適用された絞り込みだけを組み込む — 使われない絞り込みは、プランを損なう (@p IS NULL OR ...) の句ではなく、述語をまったく加えない。

ページのクエリ

```

SELECT o.id, o.order_number, o.quantity, o.due_date, o.status, o.updated_at_utc,
       p.sku, p.name
FROM production_orders o
JOIN products p ON p.id = o.product_id
WHERE o.status = ANY (@statuses)           -- only when the status filter is set
      AND o.product_id = @productId         -- only when the product filter is set
      AND o.due_date >= @dueFrom            -- only when set (inclusive)
      AND o.due_date <= @dueTo             -- only when set (inclusive)
      AND o.order_number LIKE @pattern ESCAPE '\ ' -- only when the fragment is set
ORDER BY <sort expression> <asc|desc>, o.order_number ASC
LIMIT @pageSize OFFSET (@page - 1) * @pageSize;

```

件数のクエリ — 同じ `WHERE`、`SELECT count(*)`、**結合なし**：絞り込みの述語はすべて `production_orders` にあるため、製品の結合は表示と製品の並べ替えキーにしか必要ない。002_BD M-09 の `total` とページ数を提供する。

アクセスパターン	頻度	対応するもの
最初のページ、絞り込みなし、既定の並び順（画面を開いたとき。圧倒的に最も多いリクエスト）	非常に高い	<code>ix_production_orders_due_date_order_number</code>
納期の範囲 ± 既定の並び順	高い	同じインデックス（並び順での範囲スキャン）
製品の絞り込み ± 既定の並び順	中	<code>ix_production_orders_product_id</code> （001_DB、DEC-029）の後に並べ替え
指示番号の部分一致	中	<code>ix_production_orders_order_number_trgm</code>
ステータスの絞り込み	中	専用のインデックスなし —「意図的に追加しなかったインデックス」を参照
数量、ステータス、更新日時、製品での並べ替え	低い	絞り込んだ集合に対するソートノード
一致件数	ページのリクエストごとに 1 回	上と同じ述語

並べ替えキーの対応

API の <code>sort</code> の値	SQL の並べ替え式	備考
<code>dueDate</code> （既定）	<code>o.due_date</code>	既定の方向は <code>asc</code> （DEC-002）

API の sort の値	SQL の並べ替え式	備考
orderNumber	o.order_number	形式が固定幅（PO-YYYY-NNNNN）なので、年内では辞書順が時系列順と一致する
product	p.sku, p.name	表示するラベルの順と一致する（002_BD M-06）
quantity	o.quantity	
status	CASE o.status WHEN 'Draft' THEN 1 WHEN 'InProgress' THEN 2 WHEN 'Completed' THEN 3 WHEN 'Cancelled' THEN 4 END	アルファベット順ではなくワークフロー順 — アルファベット順では Cancelled, Completed, Draft, InProgress となり、計画担当者にとって意味がない
updatedAt	o.updated_at_utc	

すべての並び順に `o.order_number ASC` を付け加える（`sort=orderNumber` のときはそれが唯一のキー）。一意なので全体の順序が決定的になり、ページ間で行が重複・欠落することはない（REQ-023）。

クエリに届くのは、アプリの許可リスト（002_BD V-13）を通ったこの 6 つの値だけで、クライアントの文字列を SQL に埋め込むことはない（`ai/rules/database.md`）。

インデックス定義

001_DB の既存のインデックスは変更しない。002_DB で新規：

インデックス名	テーブル	列	種類	根拠（アクセスパターン）
ix_production_orders_due_date_order_number	production_orders	due_date, order_number	btree	画面の既定で最も頻度の高いクエリ：納期順で指示番号による同順位の解決、任意で納期の範囲に限定。並び順と範囲の述語の両方を 1 つのインデックスで扱うため、よく使うページにはソートノードが不要
ix_production_orders_order_number_trgm	production_orders	order_number (gin_trgm_ops)	GIN (pg_trgm)	指示番号の部分一致の絞り込みは含む一致（REQ-022）で、btree では扱えない。トライグラムのインデックスで <code>LIKE '%...%'</code> がインデックスを使えるようになる（DEC-010）

`ix_production_orders_order_number_trgm` には `pg_trgm` 拡張が必要で、同じマイグレーションで作成する。`pg_trgm` は PostgreSQL 17 では信頼された拡張なので、マイグレーションの所有者のログインはスーパーユーザー権限なしで作成でき、実行時のログイン（`pmai_app`）は使うために新しい権限を必要としない。

トライグラムのインデックスについての注記：

- アプリは検索文字列を大文字にし、`ILIKE` ではなく `LIKE` で照合する：`order_number` は生成列で常に大文字（PO-YYYY-NNNNN）なので、入力を大文字にするだけで大文字・小文字を区別しない照合になり、単純な `LIKE` なら `upper(order_number)` の関数インデックスなしでインデックスを使える。

- 3 文字未満の検索文字列は完全なトライグラムを生まないため、PostgreSQL は絞り込んだ集合のスキャンにフォールバックする。これは許容する：002_BD V-09 は検索文字列を 20 文字に制限し、1〜2 文字の検索はまれで、テーブルはフォールバックが安価な程度に小さい。問題になるなら、UI で検索前に 3 文字を要求すればよい。

意図的に追加しなかったインデックス

候補	追加しない理由
<code>status</code> （単独、または <code>(status, due_date)</code> ）	値は 4 つだけで、よく使う 2 つ（ <code>Draft</code> 、 <code>InProgress</code> ）がほとんどの行を占めるため、ステータスのインデックスはすでに狭い結果のスキャンに勝つほど選択的ではない。複合の <code>(status, due_date)</code> は「ステータスの絞り込み + 既定の並び順」を扱えるが、 <code>status = ANY(...)</code> （複数選択、DEC-005）では PostgreSQL は選択した値をまたいでマージまたは再ソートしなければならない。ステータスで絞り込む一覧が主なアクセスパターンになるか、 <code>production_orders</code> がおよそ 100 万行を超えたら見直す
<code>(updated_at_utc, order_number)</code> 、 <code>(quantity, ...)</code> 、 <code>(status_rank, ...)</code>	並べ替えキーごとに 1 つのインデックスでは、画面で最も使われない並び順のために、すべての挿入と更新を遅くするインデックスが 5 つ増える。これらの並べ替えは絞り込んだ集合に対するソートノードとして実行され、想定される件数では十分に予算内である（性能の見込みを参照）
<code>(product_id, due_date, order_number)</code>	「製品の絞り込み + 既定の並び順」を 1 回のスキャンで扱えるが、001_DB が残した FK のインデックス（DEC-029）を置き換え、EF Core の規約が管理するインデックスを変えることになる。製品の絞り込みだけで集合はテーブルのおよそ 1/30 になり、その後の並べ替えはわずかなので、001_DB のインデックスの構成を変えるほどの価値はない

制約

制約の追加、削除、変更はない。001_DB のすべての制約は定義どおりのままで、001_DB がアプリで強制すると挙げたルール（納期 ≥ 今日、ステータス遷移、製品・数量のロック）も含む。Screen B は読み取り専用（DEC-003）なので、どれにも違反しない。

納期遅れの印（002_BD M-08、FN-013）はデータベースの概念ではない：`due_date`、`status`、工場ローカルの現在の日付から行ごとに計算する。格納する列でも絞り込みでもない — データベースは `due_date` と `status` を返し、アプリが判断する。これは 001_DB が納期のルールについて挙げたのと同じ理由で「今日」をスキーマの外に置くため、格納または生成した納期遅れのフラグは日付が変わった瞬間に誤りになる。

トランザクションと同時実行制御

操作	トランザクションの範囲	同時実行制御
一覧のページのクエリ + 件数（FN-010）	明示的なトランザクションなし。1 つのリクエストで 2 つの文、それぞれ <code>READ COMMITTED</code>	不要。2 つの文は異なるスナップショットを見る可能性があるため、その間に作成された行によって合計がページと 1 件ずれることがある。これはページングする一覧では普通のこと、REPEATABLE READ のトランザクションで修正せずに許容する：画面は情報表示用で、次のクエリで正され、合計が行とトランザクション的に整合していることに依存するものはない

Screen B はロックを取らず、何も書き込まない。

デモ用の初期データ

WI-004（003_DB）以降：初期データの16件のCompletedの製造指示の日付を付け直し、さらに44件のデモ用の製造指示を追加したため、デモ用のデータベースは124件の製造指示（35件Draft、25件InProgress、56件Completed、8件Cancelled）を持ち、カウンターはその後に続く。下の表は002_DBの元の80行を説明する。残りは003_DBの「デモ用の初期データ」が説明する。

DEC-007（ユーザー）は、ページング、並べ替え、絞り込み、納期遅れの印を見せられ、E2Eテストにフィクスチャーがあるよう、初期データの製造指示を必要とする。001_DBはすでに30件の製品を初期登録している。002_DBはそれらにわたって80件の製造指示を初期登録する。

特性	値
行数	80（ユーザーが求めた60～100の範囲内。既定の表示件数20で4ページ、50でも1ページを超える）
ステータスの分布	32件Draft、24件InProgress、16件Completed、8件Cancelled — どの絞り込みの組み合わせも、空でなく件数の異なる結果を返す
製品	初期登録した30製品すべてにわたり、いくつかは複数の製造指示を持つので、製品の絞り込みと並べ替えの両方に意味がある
数量	1からおよそ5,000。境界値の1と大きな値を含み、数量の並べ替えがはっきり意味を持つ
納期	マイグレーションを実行したときの工場ローカルの日付からの相対（DEC-011）：20件は過去（1～14日前）、2件は今日、58件は続く40日間。80件のうち25件は意図的に他と納期が同じで、指示番号による同順位の解決が実際に働く。適用した初期データは55種類の納期になる
納期遅れの行	過去の納期のDraftとInProgressの行 — 過去の納期の20行のうち15行 — に納期遅れの印が付く。残りの5行はCompleted / Cancelledで意図的に付かず、M-08の後半を示す
備考	ある行とない行がある（一覧は備考を表示しない。Screen Aのための初期データを現実的に保つ）。初期値はDemo order N。WI-005から、マイグレーションLocalizeDemoDataToJapanese（001_DB）がデモ用の製造指示Nに書き換える
指示番号	PO-<初期データの年>-00001 ... PO-<初期データの年>-00080を発行し、その年のproduction_order_number_countersをlast_seq = 80で初期登録するので、次にScreen Aで登録する製造指示は重複せず連番を続ける。APIで最初に登録した製造指示が00081であることを結合テストで確認する
タイムスタンプ	created_at_utc / updated_at_utc は実行日の前の数週間にずらして分布させ、「更新日時」の列とその並べ替えがすべて同じにならないようにする

この初期データの2つの特性は意図的で、はっきり述べておく価値がある：

- 納期は固定の暦の日付ではなく、実行日からの相対である（DEC-011）。固定の日付のマイグレーションは再現性が高いが、数か月後には初期データのすべての製造指示が納期遅れになり、画面のデモはそれ以降ずっと見栄えが悪くなる。相対の日付なら、環境を作り直すたびに現実的な組み合わせが保たれる。初期データのそれ以外 — id、指示番号、製品、数量、ステータス、日数のオフセット自体 — は固定なので、データは実行日が決まれば決定的で、テストは絶対の日付ではなく件数、ステータス、相対の日付で確認する。
- 初期データは空のテーブルにだけ入る。挿入はWHERE NOT EXISTS（SELECT 1 FROM production_orders）で保護されているため、実際の製造指示がすでにあるデータベースにマイグレーションを適用しても何も追加しない。したがって再実行や再適用は冪等である。

日付を実行時に計算するため、この初期データは EF Core の `HasData`（001_DB の製品の初期データのように定数の値が必要）を使えない。生成した系列に対する明示的な `migrationBuilder.Sql(...)` の `INSERT ... SELECT` として書き、これもパラメータなしで決定的である。固定の行 id は `ProductSeed` と同じ形式の UUIDv7 の形の定数。

アプリのデータベース権限

変更なし。Screen B に必要なのは `production_orders` と `products` の `SELECT` だけで、`pmai_app` はすでに持っている（001_DB、DEC-016）。本画面では `production_order_number_counters` の権限は不要で、新たに得ることもない。`pg_trgm` の演算子クラスを使うのに権限付与は不要。

拡張、インデックス、初期データを作るマイグレーションは、すべてのマイグレーションと同様に**所有者**のログインで実行する（`deploy/README.md`）。

API/DD との対応

DD の項目名は 002_BD §3 の変数名。API の項目名は 002_DD-API で確定する。一覧のレスポンスは、列を持たない計算済みのレスポンス専用の項目を 1 つ追加する。

テーブル.列	DD の項目	API の項目
<code>production_orders.id</code>	（行のリンク先）	<code>id</code>
<code>production_orders.order_number</code>	行の指示番号／ <code>orderNumber</code> の絞り込み	<code>orderNumber</code> （行）、 <code>orderNumber</code> （クエリ：部分一致）
<code>production_orders.product_id</code>	<code>productId</code> の絞り込み	<code>productId</code> （クエリ）
<code>products.sku</code> 、 <code>products.name</code>	行の製品のラベル（M-06）	<code>product.sku</code> 、 <code>product.name</code>
<code>production_orders.quantity</code>	行の数量	<code>quantity</code>
<code>production_orders.due_date</code>	行の納期、 <code>dueFrom</code> / <code>dueTo</code> の絞り込み	<code>dueDate</code> （行、ISO YYYY-MM-DD）、 <code>dueFrom</code> 、 <code>dueTo</code> （クエリ）
<code>production_orders.status</code>	行のステータス、 <code>status</code> の絞り込み	<code>status</code> （行）、 <code>status</code> （クエリ、繰り返し可）
<code>production_orders.updated_at_utc</code>	行の最終更新日時	<code>updatedAt</code> （ISO 8601 UTC）
—（ <code>due_date</code> 、 <code>status</code> 、工場の今日から計算）	行の納期遅れの印（M-08）	<code>isOverdue</code> （レスポンスのみ）
—（件数のクエリで計算）	結果の概要（M-09）	<code>total</code> （レスポンスのみ）
—（クエリの操作値をそのまま返す）	並び順、方向、ページ、表示件数	<code>sort</code> 、 <code>dir</code> 、 <code>page</code> 、 <code>pageSize</code>
<code>production_orders.notes</code> 、 <code>created_at_utc</code> 、 <code>order_year</code> 、 <code>order_seq</code> 、 <code>xmin</code>	—	一覧のエンドポイントでは公開しない

性能の見込み

インデックスの選択を好みの問題ではなく確認できるものにするための規模の前提：1 つの工場が 1 日に 50 件程度の製造指示を登録すると、1 年でおよそ 15,000 行になるため、テーブルは何年も数十万行の前半にとどまる。デモ用のデータベースは 80 行。

クエリ	見込み
既定のページ（絞り込みなし）	<code>ix_production_orders_due_date_order_number</code> のインデックススキャンで、 <code>pageSize</code> 行 + オフセットで止まる。ソートノードなし
納期の範囲 + 既定の並び順	同じインデックス、範囲を限定したスキャン
指示番号の部分一致	トライグラムのインデックスのビットマップインデックススキャンの後に絞り込み
製品の絞り込み	<code>ix_production_orders_product_id</code> のインデックススキャンの後に、テーブルのおよそ 1/30 の並べ替え
ステータスの絞り込み、または既定以外の並べ替えキー	絞り込んだ集合のスキャン + ソートノード — 上の規模では許容範囲
件数	同じ述語。画面は正確な合計を報告するため（REQ-024）、推定ではなく正確な <code>count(*)</code>

テーブルがおよそ 100 万行を超えた場合、深いページング（大きな `pageSize` での大きな `page`）が一般的になった場合 — `OFFSET` は線形に増えるため、画面が定める「`{n} / {m}` ページ」の操作を犠牲にしてキーセットによるページングのほうが望ましくなる — または、ステータスの絞り込みが主なアクセスパターンになった場合は、この設計を見直す。

マイグレーションの影響と復旧の限界

この順で 2 つのマイグレーション：

1. `AddProductionOrderListIndexes` — インデックスと拡張。

- **マイグレーションの種類**：追加のみ。`CREATE EXTENSION IF NOT EXISTS pg_trgm`、その後に両方のインデックス。
- **同時実行**：`production_orders` はすでに使用中なので、`ai/rules/database.md` に従い、両方のインデックスを `CREATE INDEX CONCURRENTLY IF NOT EXISTS` で作成する。`CONCURRENTLY` はトランザクション内では実行できず、EF Core はマイグレーションをトランザクションで包むため、これらの文は `migrationBuilder.Sql(..., suppressTransaction: true)` で発行する。実行前にその結果を理解しておく必要がある：マイグレーションは**アトミックではない**。途中で失敗すると PostgreSQL は `INVALID` のインデックスを残し、それはどのクエリにも使われず、マイグレーションを再試行する前に削除（`DROP INDEX CONCURRENTLY`）しなければならない。`SELECT indexrelid::regclass FROM pg_index WHERE NOT indisvalid;` で確認する。
- **データ復旧の限界**：なし — データを書き込んだり変更したりしない。
- **ロールバック計画**：両方のインデックスを `DROP INDEX CONCURRENTLY IF EXISTS` で削除する。拡張はそのまま残す（削除は不要で、それを使う他のオブジェクトを壊す）。このマイグレーションを戻しても一覧が遅くなるだけで、誤りにはならない。

2. SeedDemoProductionOrders — デモ用のデータ。

- **マイグレーションの種類**：データの挿入、保護付き。production_orders が空のときだけ、production_orders の 80 行と production_order_number_counters の 1 行を挿入する。
- **データ復旧の限界**：初期データは空のテーブルにだけ書き込むので、適用しても何も失われない。戻すと初期データの 80 行を**固定の id** で削除し、ユーザーが Screen A で初期データの製造指示に加えた編集も削除される — それらの編集はマイグレーションからは復旧できない。ユーザーが登録した製造指示は、id が初期データの集合にないため影響を受けない。
- **ロールバック計画**：DELETE FROM production_orders WHERE id IN (<80 個の固定の id>) とし、その年のカウンターの行をリセットする。初期データの後に作成された行は影響を受けない。
- **運用上の注記**：納期は実行日から計算するため、後日に新しいボリュームでこのマイグレーションを再適用すると異なる納期になる（設計どおり、DEC-011）。したがってテストの確認は、絶対の日付ではなく、件数、ステータス、今日からのオフセットで書かなければならない。

どちらのマイグレーションも所有者のログインで実行し、pmai_app の権限の変更は不要。

どちらも 2026-09-22 にローカルの Compose のデータベースに適用した：2 つのインデックスが存在し、pg_index は INVALID のインデックスを報告せず（アトミックでない CONCURRENTLY の経路が完了した）、テーブルは 55 種類の納期にわたる 80 件の初期データの製造指示を持つ。

初期データは結合テスト用のデータベースにも入る。そこは使い捨ての PostgreSQL のコンテナに対して同じマイグレーションを実行する。これは意図的である — 一覧のテストには現実的なデータが必要で、実際のマイグレーションの一式をテストすることが目的である — が、そこでの指示番号は初期データの 80 件の後から始まることになる。その結果は WI-003 DEC-012 に記録している。

未決定事項

決定事項	選択肢	推奨	状態
大文字・小文字を区別しない指示番号の部分一致検索のインデックス	pg_trgm の GIN インデックス／前方一致だけの upper(order_number) の btree／インデックスなし	pg_trgm の GIN — REQ-022 の「含む」の意味をインデックスで支える唯一の選択肢	決定 — work-items/WI-003/decisions.md DEC-010
初期データの納期：固定の暦の日付か、マイグレーションの実行日からの相対か	固定（ HasData 、完全に再現可能）／相対（ INSERT ... SELECT 、いつまでも現実的）	相対。それ以外はすべて固定し、挿入は空のテーブルに限定する	決定 — DEC-011
ステータスの絞り込みの専用インデックス	(status, due_date) ／なし	今はなし。約 100 万行を超えるか、ステータスの絞り込みが主になったら見直す	決定 — ここに記録。個別の DEC は不要
OFFSET の代わりにキーセットによるページング	キーセット（深いページが速い、ページ番号なし）／オフセット（ページ番号あり、線形のコスト）	オフセット — 画面は「{n} / {m} ページ」を定め（REQ-024）、件数はキーセットを正当化しない	決定 — ここに記録